Development of a Heterogeneous Reconfigurable CGRA Cell

Duarte Sanchez David, Jimenez Ulate Eddy, Ruiz Rivera Jeffrey, Esquivel Arias Fabian, Zúñiga Ramírez Jose Eduardo
School of Electronics Engineering
Costa Rica Institute of Technology
{davidduarte28, e.jimenez.10, yr1ruiz, fabiesqui22, je.zuniga}@estudiantec.cr

***Abstract*—Coarse-Grained Reconfigurable Arrays (CGRAs) are gaining interest in the industry due to the flexibility they provide and the facility to program them. There different architectures of CGRAs, being the most common the homogeneous CGRA but these present a problem of underutilization of resources when the workload is not uniform. This works presents a heterogeneous CGRA cell architecture composed of five different elements: a routing cell, distribuited memory cell, scalar processing element (PE), vector PE and a MAC PE. The design was implemented in SystemC and validated with Vitis HLS, using two main kernels: GEMM and sum-reduction. For GEMM, the spatial mapping reaches higher throughput (33.74 MOP/s vs 13.56 MOP/s) and lower latency per operation (29.64 ns/op vs 73.72 ns/op), at the cost of higher resource usage (up to 4.58× FF and 4.00× DSP). For sum-reduction, the temporal MAC-based mapping achieves better efficiency, with higher utilization (79.7% vs 47.8%) and lower latency (50.95 ns/op vs 55.65 ns/op), while also requiring fewer resources. These results support the central claim that heterogeneous, workload-aware CGRA composition improves performance-resource trade-offs compared with uniform PE deployment.**

***Index Terms*—coarse-grained reconfigurable architecture (CGRA), heterogeneous CGRA, spatial CGRA, reconfigurable computing.**

I. Introduction

The reconfigurable architectures have been very successful in industry because software evolution has advanced rapidly, with diverse applications, different user needs, and scientific progress, so hardware cannot adapt at the same pace as software [1]. Special-purpose systems can suffer from rapid obsolescence, and if the price of hardware development and production is high, it becomes economically unviable [1].

Currently, architectures such as FPGAs (Field Programmable Gate Arrays) have been widely adopted in diverse applications due to their flexibility and customization capability, gaining traction in HPC (High-Performance Computing). However, FPGAs present certain limitations such as slow configuration times, long compilation times, and in cases, low operating frequencies [2].

In the recent years there has been growing interest in HPC in using architectures such as CGRAs (Coarse-Grained Reconfigurable Arrays), which offer an intermediate solution between FPGAs and ASICs (Application Specific Integrated Circuits). CGRAs are hardware architectures that, conceptually, allow silicon to be malleable and their functionality to be dynamically reconfigured [2]. CGRAs have the advantage of being one to two orders of magnitude more efficient than FPGAs and more than 2 to 3 orders of magnitude more efficient than ASICs [1].

The development of a reconfigurable CGRA cell is of great importance not only for its relevance in HPC, but also for its adoption in Machine-Learning applications, image processing, and computer vision, communications, and other areas [3]. Therefore, the objective of this work is to develop a heterogeneous CGRA with different kind of processing elements such as a scalar and vector processing cells, memory banks and a MAC (Multiply-Accumulate) unit.

II. Background and Related Work

In recent years, CGRAs have gained significant prominence as hardware accelerators due to their ability to offer an exceptional balance between software flexibility and the high performance of application-specific hardware [1], [2]. Unlike fine-grained architectures, CGRAs operate at the word or block level, which substantially reduces reconfiguration and routing overhead, making them particularly attractive for efficient edge-computing applications. The development and research of these architectures has been strongly driven by the adoption of the open-source RISC-V instruction set architecture (ISA) [4], whose extensible nature facilitates the creation of highly customized heterogeneous SoCs (Systems on Chip).

At the level of practical implementation, these architectures are the result of cutting-edge open-source efforts. Repositories such as risc-v-cgra (developed by ECASLab) investigate these integrations at the system level. OpenCGRA is a unified framework that supports the full top-to-botton design flow, this adopts a Python-based PyMTL3 framework to model the architecture and enabling the RTL generation and muti-level simulation [5] TRAM is another example which adopts a Chisel-based CGRA template to model hardware architecture with register transfer level (RTL) code generation and functional-level (FL) simulation; TRAM proposes a flexible and scalable CGRA template composed of GPEs, IOBs and GIBs with two-dimensional (2D) island-style topology [6]. HETA is a temporal CGRA implementing a heterogeneous architecture with a Bayesian optimization, this CGRA is a 2D island style layout build from four type of blocks: GPE (Generic Processing

Element), GIB (General Interconnect Block), LSU (Load/Store Unit) and IOB (I/O Block) [7].

For this type of system to reach its full potential, the internal design of the individual reconfigurable cell (PE) and its associated compilation tools are critical [8]. They highlight the need for context-memory-aware mapping to ensure energy-efficient acceleration, emphasizing that data-access latency dictates overall performance. Within this framework, the design and development of a new reconfigurable CGRA cell is grounded in the need to optimize local interconnects, deterministic access to intermediate operands, and the versatility of its cycle-level operations — challenges that remain areas of active exploration at the frontier of heterogeneous accelerator design.

III. Analysis of Existing Solutions (SOTA)

Nowadays there are different types of CGRAs, you can classify them by their elements in heterogeneous or homogeneous, also depending on how you use structure the hardware and the handling of the data, they can be classified as temporal or spatial. Examples of homogeneous CGRAs are the HyCUBE, DySER, TRAM and OpenCGRA [5], [6], [9] which have an array of homogeneous or equal PEs, while examples of heterogeneous CGRAs are Plasticine and HETA [7], [10] which combine different PE.

CGRA systems are no longer seen as an isolated element; today they are presented as a heterogeneous part of an architecture system [3]. Furthermore, CGRAs have become highly important in HPC (High Performance Computing) systems, which greatly benefit from having heterogeneous-type PEs, with a mix of basic and complex PEs [11]. This is highly important for mathematical functions, since traditional CGRAs do not handle them optimally.

In the table I it is shown a comparison between four types of CGRAs. They present different trade-offs, where they are developed according to the area of application or workload. For example the HETA CGRA is good when the physical space is a concern [7] or the Plasticine is a good option when the workload is highly parallel [10].

Table I
Comparison of state-of-the-art solutions

Solution	Strengths	Weaknesses	Viability	PE / CGRA Type
HETA [7]	Bayesian DSE requires zero manual intervention, very fast CGRA.	Only functional and RTL simulation, limited veification tooling.	Good for research, ideal for temporal CGRA exploration.	Heterogeneous / Temporal.
TRAM [6]	High PE utilization and clean interconnect model.	Spatial CGRA only, cannot time-multiplex resources.	Good for research, when the interconnect flexibility and mapping are priorities.	Homogeneous / Spatial.
OpenCGRA [5]	Very complete, with multi-level simulation. Based on mature LLVM and PyMTL3 infrastructure.	The mapper can give a low PE utilization. Since it uses python-based modeling, large designs can be slower.	Ideal for research and prototyping, presenting a very complete verification and integration flow.	Homogeneous / Spatial.
Plasticine [10]	Massive per-cycle processing (OP/s), built for parallel patterns.	Rigid programming and restrictive power consumption.	Fixed infrastructure/servers, proved at 28nm with real performance numbers.	Heterogeneous / Spatial.

IV. Proposed Solution and Second-Level Architecture

Architectures like the homogeneous CGRA present a challenge where there is under-utilization of resources like PEs and interconnects [12], [13]. Since all PEs are identical and have general-purpose capabilities, a large fraction of functional units remain idle during execution [7]. In addition, the uniform interconnects can leave many routes unused [3]. This kind of architecture has the advantage of being simple to design and implement, but that same simplicity is a limitation when high parallelism and throughput are required. Such architectures are not energy-efficient and have low performance [9].

In contrast, heterogeneous CGRAs have different types of PEs, which allows a better utilization of the resources like the HETA architecture [7], however even in this case still exists a limitation, where the PEs are not specialized enough and are limited to a specific workflow. For example the HETA architecture supports scalar operations but doesn't mention the support for vector operations [7], on the other hand, the Plasticine architecture supports vector operations but doesn't mention the support for scalar operations [10].

This work presents a heterogeneous CGRA with different PE types, routing cells and distributed memory cells. This design is proposed in order to have specialized cells according to the necessity of the workload, having a specialized element for a wider range of operations with less elements. The elements of the CGRA are as follows:

- **Routing:** statically-configured combinational switch (4 stored contexts).
- **Distributed memory:** local dual-port SRAM banks (2 KB per cell in the current model, parametrizable).
- **Scalar processing unit:** fixed 32-bit RV32I+MUL ALU with a single 16-instruction program.
- **Vector processing unit:** int32 SIMD ALU with 4 fixed lanes.
- **MAC processing unit:** multiply-accumulate unit with a persistent per-lane accumulator for high-throughput operations.

With these elements, we are introducing a new heterogeneous CGRA architecture that can be used for scalar operations and vector operations, with a better utilization of the resources and a better performance. Currently no other architecture has been found that combines scalar and vector operations, neither with routing cells and distributed memory cells, which makes this architecture unique in its kind.

A. Architectural Proposal

This architecture proposal is shown in the figure 1, where the different elements of the CGRA are present. In this case is a basic array of 2x3 but this could be expanded according to the necessity.

1) Routing Cell: Implemented as a purely combinational switch: 8 ports (4 mesh + 4 local), each output selecting any of the 8 inputs (or none) via a per-context 4-bit selector. Concurrent readers of the same source are served as copies

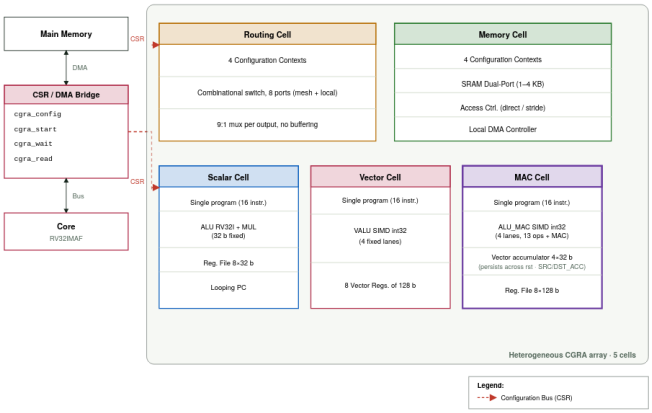

Figure 1. Second-level architecture diagram generated with artificial intelligence.

rather than arbitrated, giving inherent multicast fan-out. Four contexts are stored per cell, each loaded in a single clock edge; which context is active is chosen combinationally.

2) Distributed Memory Cell: Dual-port SRAM (2 KB per cell in the current instantiation, configurable via a template parameter); direct and strided access modes ; a local DMA engine performs host-free block transfers between SRAM and the NoC. Four independently configurable access contexts are supported. This design still eliminates centralized memory contention by giving each cell its own local bank.

3) Scalar Cell: Integraded by a 32-bit ALU (RV32I integer/logic ops plus MUL) with a 8 $\times$ 32 b register file and a single 16-instruction program memory that the program counter loops through.

4) Vector Cell: A 4-lane int32 SIMD VALU (add, sub, logic, shifts, compare, MUL, the same op set as the Scalar ALU, applied per lane) with 8 $\times$ 128 b vector registers and a single 16-instruction program memory.

5) MAC Cell: This PE uses a VALU op set and 8 $\times$ 128 b register file and a dedicated MAC opcode. Each lane's product is computed the same way as MUL, then added in the writeback stage into a persistent per-lane accumulator (4 $\times$ 32 b) that survives `rst` and is addressable like any other operand via dedicated source/destination selectors.

B. Development Tools and Environment

The development of this heterogeneous CGRA starts with the implementation of SystemC to verify the functionality of the architecture and its elements. Afterwards, the SystemC model is translated so it can be used in Vitis 2024.1 and generate a RTL code and get the metrics of the design. Also there will be a sofware validation using GEM5 to simulate the RISC-V core and the CSR/DMA interface, while the CGRA will be modeled in SystemC.

In order to ensure strict reproducibility of the design and simulation tests, the software tooling infrastructure and its respective versions are defined in the table II.

Table II
DEVELOPMENT TOOLS AND ENVIRONMENT VERSIONS

Tool / Software	Version	Purpose in the Experiment
SystemC	3.0.1	Modeling and simulation of the CGRA architecture functionality.
Vitis HLS	2024.1	High-Level Synthesis (HLS) from SystemC to RTL (VHDL).
GEM5	v25.1	Simulation of the RISC-V core and CSR/DMA interface, integrated with the SystemC CGRA model.

V. EXPERIMENTAL PLAN AND METHODOLOGY

A. Testing and Stimulus

To test an electronic design, a testbench is needed, in charge of driving a stimulus into the design and facilitating monitoring of what happens in the design under test (DUT). As has been done in other designs, and in the various CGRA development frameworks, it becomes necessary to have an algorithm that the architecture is capable of executing. For this, one typically starts from so-called kernels: representative algorithms, typically coded in C (for example, Sum-Reduction and GEMM), annotated with compiler directives on their loops, whose data-flow structure can be mapped onto a CGRA's PE array. This is, in fact, the approach followed by Morpher within the OpenCGRA ecosystem: starting from the C kernel, it generates the data-flow graph (DFG) and the scratchpad memory layout, and produces the test vectors that feed the simulation/emulation stage, which are used to verify functional correctness and obtain area and power estimates [14]. The use of Sum-Reduction as a reference kernel is also consistent with the CGRA literature oriented toward signal processing, where it recurrently appears alongside routines such as IDCT, FIR, and IIR [2].

It is worth noting that even in well-established frameworks such as OpenCGRA, the authors themselves point out that the simulator relies on testbenches manually written by the user and does not automate test-data generation or end-to-end verification [14]. This reinforces the relevance of proposing, from the outset, a structured and progressive testing flow such as the one proposed below.

While a valuable goal is to achieve end-to-end execution of an algorithm such as Sum-Reduction or GEMM compiled from start to finish (from the C program to the configuration bitstream and the CGRA's input data), given the time available for the project's development, the first objective targets algorithmically simpler tests. The following steps are proposed:

1) **A single PE executing $c = a+b$ (constant) or $c = a \times b$.** This validates loading of the configuration word (config word), reading of registers/inputs, the ALU operation, and writing of the output result. It is, literally, the architecture's "hello world."
2) **Chains of operations, in pipeline mode:** accumulated sum ($y = x_0+x_1+x_2+\ldots+x_n$) and successive-squaring powers ($x,\ x^2,\ x^4,\ x^8,\ \ldots$).
3) **Simple MAC ($acc\ +=\ a \times b$):** this is the base block of almost every real kernel (Sum-Reduction, GEMM,

convolutions), with this working correctly, the validation can scale toward more complex designs.

Once these basic tests are passed, progress can be made toward typical benchmarks from the CGRA literature, such as MachSuite and PolyBench [15], [16], which together with Wavelib and BEEBS make up the set of suites used by Morpher to evaluate its compilation flow [14]. These suites provide kernel libraries (including Sum-Reduction and GEMM) that can be mapped onto the CGRA through a compilation flow similar to the one described above.

For the software validation it is planed to use the softmax kernel. This will be executed using GEM5 to simulate the interaction between a RISC-V core and the CGRA.

B. Monitoring

This project will be developed in SystemC, the aim is to leverage the language's own features to implement performance measurement. It is proposed to implement a monitor as an `SC_MODULE` component that observes, non-intrusively, DUT elements such as signals and internal counters. This separation between functional logic and measurement logic follows SystemC's own modular design principle, in which the monitor can be added to or removed from the testbench without modifying the module under analysis.

It is proposed that the monitor compute, at minimum, the following metrics when evaluating the HDL post-synthesis:

- **Operations per second (OP/s):** ratio between the total number of operations executed by the DUT and the total execution time.
- **Average latency:** recording `sc_time_stamp()` at the start and end of each individual operation, averaged over the sampling window.
- **Utilization rate:** proportion of active cycles relative to the total number of cycles.

These metrics are printed to the console via `SC_REPORT_INFO`.

Additionally, other metrics that will be taken into account are the quantity of:

- LUT (Look-Up Table)
- FF (Flip-Flops)
- DSP (Digital Signal Processing)
- BRAM (Block RAM)
- SRL (Shift Register LUT)

These are necessary to measure the resource utilization of each design and are obtained from Vitis 2024.1 after the synthesis of the design.

Regarding the performance metrics when trying to evaluate the software integration with the CGRA, the following metrics will be taken into account for the specified kernel (softmax function):

- **Test cases passed:** number of test cases that passed successfully.
- **Worst per-lane error (LSB):** the worst-case error in least significant bits (LSB) across all lanes.
- **Maximum sum deviation (LSB):** the maximum deviation from the expected sum in LSB.
- **Output probability sum range:** the range of the sum of output probabilities, which should be close to 1 for valid softmax outputs.

C. Architectural Exploration

Tied to design-space exploration, the plan is to generate variations of the CGRA's architectural design by modifying the number and type of PEs (*processing elements*). For example, changes in the distribution of PE types (more memory banks, more fixed-point ALUs, more floating-point ALUs), as well as different $n \times n$ array dimensions, aimed at optimizing the execution of vector instructions. This type of exploration is consistent with the diversity of CGRA architectures reported in the literature, where performance depends heavily on the composition and arrangement of the PEs [2].

VI. Results and Discussion

This section presents the results obtained from the implementation of the CGRA with the proposed kernels and monitoring metrics from section V.

A. GEMM Kernel

The GEMM (General Matrix-Matrix Multiplication) kernel was implemented in two ways using the MAC cell, the temporal implementation and spatial implementation. The first one uses a single MAC cell to perform the operation, while the other one uses 4 MAC cells in total to do the same operation. The table III shows the resource utilization of both implementations, where as expected, the temporal implementation is cheaper in resources than the spatial implementation.

Table III
Post-synthesis resource utilization (Vivado) for temporal vs. spatial implementations.

Resource	Temporal	Spatial	Spatial/Temporal
LUT	1199	4467	3.73×
FF	1551	7098	4.58×
DSP	3	12	4.00×
SRL	0	0	–
BRAM	0	0	–

Regarding the performance of each configuration, the table IV shows that the spacial implementation has a better performance than the temporal implementation. The quantity of operations per second is higher in the spatial implementation and has a lower latency per operation, since the PEs are the same, the spatial has the advantage because it has more parallelism available.

The downside of the temporal implementation is the necessity to have a start=true transaction for each invocation, while the spatial implementation only needs one invocation. This represents a total of 204 cycles vs 82 cycles of the spatial implementation, in the table V are the values of each implementation.

Table IV
PERFORMANCE METRICS FROM MEASURED 2×2 GEMM EXECUTION.

Metric	MAC Cell - Temporal	MAC Cell - Spatial	Spatial/Temporal
Operations per second (MOP/s)	13.56	33.74	2.49×
Average latency per operation (ns/op)	73.72	29.64	0.40×
Utilization rate (%)	58.6	71.9	1.23×

Table V
PER-TRANSACTION RTL CYCLE COUNTS FROM COSIMULATION (ACTUAL SIMULATED HARDWARE).

Design	# prog_valid transactions	Cost each	start=true transaction cost	Outputs per start=true
Temporal	3	12 cycles	**51 cycles**	1 ($C[i][j]$)
Spatial	16	2 cycles	**82 cycles**	4 (whole matrix)

B. Sum-Reduction Kernel

For the sum-reduction kernel we did two comparison, one temporal implementation with our new MAC cell and other spatial implementation with 4 scalar cells. Both of the solutions were implemented under the same hardware considerations, using a Kria K26 SoM as reference with a 250 MHz clock target. The setup for this validation is shown in the table VI.

Table VI
TEMPORAL VS. SPATIAL SUM-REDUCTION SETUP.

Item	Temporal	Spatial
Mesh	`MAC PE`	`Scalar PE` × 4
Cell count	1	4
`NUM_PHASES`	8	1
`INSTR_MEM_SIZE`	3	3
Test vectors	`{6,-2,9,4,0,7,-5,3}` = 22, `{100,-55,30,7,-12,4,-9,15}` = 80	same

The results of these implementations are shown in the table VII. The new MAC cell shows a better utilization compared with the standard solution given by the literature, like the TRAM solution [6].

Table VII
POST-SYNTHESIS RESOURCE UTILIZATION FOR SUM-REDUCTION KERNEL.

Resource	MAC PE - Temporal	Scalar PE - Spatial	Spatial/Temporal
LUT	1205	3857	3.20×
FF	1551	5230	3.37×
DSP	3	12	4.00×
SRL	0	20	–
BRAM	0	0	–

Regarding the performance metrics, the table VIII shows that the temporal implementation with the MAC cell shows a better performance since this kind cell is designed to perform the sum-reduction operation in a more efficiente way that a normal scalar cell. With this results is possible to confirm the importance of having a heterogeneous CGRA with specialized cells for specific operations, since this allows to have a better performance and a better resource utilization.

Table VIII
PERFORMANCE METRICS FROM MEASURED SUM-REDUCTION EXECUTION.

Metric	MAC PE - Temporal	Scalar PE - Spatial	Spatial/Temporal
Operations per second (MOP/s)	19.63	17.97	0.92×
Average latency per operation (ns/op)	50.95	55.65	1.09×
Utilization rate (%)	79.7	47.8	0.60×

1) Softmax Kernel: For the validation of the softmax kernel, the proposed heterogeneous CGRA was tested using GEM5 to simulate the RISC-V core and the CSR/DMA interface, while the CGRA was modeled in SystemC. In this validation, the vector unit performed the parallel reduction, exponential, and normalization stages, while the scalar unit computed the reciprocal term $1/S$ using a Newton–Raphson approximation. There were six test cases executed, with different

The softmax kernel was also validated end-to-end on the proposed heterogeneous CGRA. In this experiment, the vector unit performed the parallel reduction, exponential, and normalization stages, while the scalar unit computed the reciprocal term $1/S$ using a Newton–Raphson approximation. Six representative test cases were executed with different test cases like uniform inputs, dominant-lane cases, negative fractional inputs, and a saturation scenario with input spread beyond the useful exponential range. In the table IX the summary of the validation is shown, where all the test cases passed successfully, with an error below the acceptance threshold

Table IX
VALIDATION SUMMARY FOR END-TO-END SOFTMAX EXECUTION.

Metric	Observed	Acceptance / Comment
Test cases passed	6/6	All passed
Worst per-lane error (LSB)	2.81	< 6.00 LSB
Maximum sum deviation (LSB)	4.00	≤ 8.00 LSB
Output probability sum range	0.999023 – 0.999512	Close to 1
Ordering preserved	Yes	In all test cases
Dominant-lane case	Passed	Stable behavior
Saturation case	Passed	Stable under large spread

These results show that the proposed heterogeneous organization can correctly support more complex nonlinear kernels beyond GEMM and sum-reduction, while maintaining good numerical accuracy under both nominal and extreme input conditions.

VII. CONCLUSIONS

In this work we propose a heterogeneous CGRA with diffent types of cells, including a new MAC PE that haven't been mention in past works. This work shows that having different types of cells in a CGRA architecture can translate in a better performance. The design of a CGRA depends on the type of workload that is going to be executed, but for general purposes the heterogeneous architecture with specialized cells present a better solution for performance. Also the type of

architecture: spatial or temporal is very important to take into consideration because of the space available to implement the GGRA, the spatial is going to use more resources. It is important to confirm not only the hardware functionality but also the software integration with the CGRA, since this is a key point to have a complete solution.

VIII. Future Work

This work is first the first step in the development of a heterogeneous CGRA, so there is still work that can be done to improve the architecture and validation of the design. The architecture can be increased in complexity by implementing a large number of PEs, this needs a more complex communication between the different cells but improving the capacity of the CGRA. Also adding more variety of PEs can improve the performance of the CGRA, since depending of the workload, the more suitable PE can be used to improve the performance. In terms of validation, the CGRA can be tested with more complex kernels and also can be emulated in a FPGA to get more accurate results.

AI Tool Usage Declaration

For this work, Artificial Intelligence was used to help structure the document, as well as for a grammar and spelling review of the document. The tables were also generated using AI tools aswell as the figures.

References

[1] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. H. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Computing Surveys*, vol. 52, pp. 1 – 39, 2019.

[2] K. S. Artur Podobas and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE ACCESS*, vol. 8, 2020.

[3] G. Zhu, L. Deng, K. Zhang, W. Fan, B. Jin, W. Cao, F. Zhang, X. Zhou, F. Zhang, and X. Yu, "DVHetero: A framework for designing and validating heterogeneous SoC with RISC-V processor and CGRA," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 3, 2025.

[4] EdgeIR Staff. (2024, jan) What is risc-v and why is it important? online. Accessed: 2026-06-11. [Online]. Available: https://www.edgeir.com//what-is-risc-v-and-why-is-it-important-20240109

[5] C. Tan, N. B. Agostini, J. Zhang, M. Minutoli, V. G. Castellana, C. Xie, T. Geng, A. Li, K. Barker, and A. Tumeo, "Opencgra: Democratizing coarse-grained reconfigurable arrays," in *2021 IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, 2021, pp. 149–155.

[6] Y. Qiu, Y. Cao, Y. Dai, W. Yin, and L. Wang, "Tram: An open-source template-based reconfigurable architecture modeling framework," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*, 2022, pp. 61–69.

[7] Y. Dai, J. Li, Q. Zhu, Y. Qiu, Y. Hu, W. Yin, and L. Wang, "Heta: A heterogeneous temporal cgra modeling and design space exploration via bayesian optimization," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 32, no. 3, pp. 505–518, 2024.

[8] S. Das, K. J. M. Martin, and P. Coussy, "Context-memory aware mapping for energy efficient acceleration with cgras," *Design, Automation & Test in Europe Conference & Exhibition*, vol. null, pp. 336–341, 2019. [Online]. Available: https://www.semanticscholar.org/paper/b3c002d29c59700cc1827aff29a72b560c86346a

[9] M. Wijtvliet, H. Corporaal, and A. Kumar, *Blocks, Towards Energy-efficient, Coarse-grained Reconfigurable Architectures*. Springer Nature Switzerland AG, 2022.

[10] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel patterns," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, Toronto, ON, Canada, 2017.

[11] E. Del Sozzo, X. Wang, B. Adhi, C. Cortes, J. Anderson, and K. Sano, "Exploration of Trade-offs Between General-Purpose and Specialized Processing Elements in HPC-Oriented CGRA," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*. IEEE, 2024.

[12] Mei, Bingfeng and Vernalde, Serge and Verkest, Diederik and De Man, Hugo and Lauwereins, Rudy , "ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix," in , 2004, kU Leuven.

[13] Karunaratne, Manupa and Kulkarni, Aditi and Mitra, Tulika and Peh, Li-Shiuan , "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in , 2017, nTU Singapore.

[14] R. Juneja, P. Dangi, T. K. Bandara, Z. Li, D. Wijerathne, L.-S. Peh, and T. Mitra, "Building an open cgra ecosystem for agile innovation," in *Proceedings of the IEEE//ACM International Conference on Computer-Aided Design (ICCAD)*, 2025, pp. 1–9. [Online]. Available: https://arxiv.org//abs//2508.19090

[15] L.-N. Pouchet, "PolyBench/C: The polyhedral benchmark suite," https://www.cs.colostate.edu/~pouchet/software/polybench/, 2012, accessed: 2026-06-18.

[16] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, "MachSuite: Benchmarks for accelerator design and customized architectures," https://github.com//breagen//MachSuite, 2014, gitHub repository. Accessed: 2026-06-18.